

1. Johdanto

Kaukojunaliikenne ja sen aikataulut ovat suuri osa suomalaisten arkea ja liikkumista. Junia käyttävät päivittäin sadat ihmiset, ja junien myöhästymiset aiheuttavat monia erilaisia haittoja: aikataulujen sekoamista, vaihtoyhteyksien menettämistä ja matkustajien tyytymättömyyttä. Vaikka junien tietoja kerrotaan VR:n ja Fintrafficin sivuilla, eivät matkustajat pysty ennustamaan ovatko tietyt junat myöhässä ennen kuin ne ovat jo myöhässä.

Tämän projektin taustalla on idea koneoppimismallista, joka voisi auttaa ennustamaan junan myöhästymisen todennäköisyyttä etukäteen, heti junan lähdettyä, hyödyntäen historiallista aikataulu- ja toteumadataa. Tämä voisi auttaa asiakkaita arvioimaan esimerkiksi jatkoyhteyksiin ehtimistä paremmin.

Tässä raportissa on kyseessä koneoppimisen luokitteluongelma, jossa tavoitteena on ennustaa, myöhästyykö tietty juna vähintään tietyn kynnsarvon (5 minuuttia) verrattuna aikatauluun, käyttäen Fintrafficin Digitraffic-rautatieliikenne-rajapinnan avointa historiadataa.

Raporttimme sisältö on jaettu lukuihin. Luvussa 2 määrittelemme luokitteluongelmamme. Aineisto ja mallit kuvataan luvussa 3. Tähän kirjoitetaan myös mitä muut luvut pitävät sisällään.

2. Ongelman muodostaminen

Muodostamme junien myöhästymisen ennustamisen binääriseksi luokitteluongelmaksi (binary classification). Tavoitteena on ennustaa, saapuuko tietty juna pääasemalle vähintään ennalta määritellyn kynnsarvon (esim. 5 minuuttia) aikataulusta jäljessä. Käytämme valvottua oppimista (supervised learning), jossa mallia koulutetaan aikaisemmalla junadatala, jossa jokaiselle havainnolle tiedetään sekä piirteet että todellinen lopputulos (oliko juna yli kynnsarvon myöhässä vai ei).

Jokainen datapiste on junamatkaa kuvaava JSON-objekti, joka sisältää tmm. tietoa itse junasta (`trainNumber`, `departureDate`, `trainType`, `trainCategory`), tilatiedot (`cancelled`, `runningCurrently`) sekä listan `timeTableRows`, joka sisältää jokaisen aseman kohdalla saapumis- ja lähtörivin. Jokainen rivi sisältää mm. aseman koodin (`stationShortCode`), aikataulun mukaisen ajan (`scheduledTime`), toteutuneen ajan (`actualTime`) sekä niiden erotuksen minuutteina (`differenceInMinutes`). Lisäksi asemasta on tieto siitä, pysähtyykö juna asemalla (`trainStopping`) ja onko pysähdys tarkoitettu matkustajille (`commercialStop`).

Yksi datapiste vastaa yhtä junamatkaa lähtöasemalta pääasemalle. Piirteinä käytämme lähtöhetken myöhästymää, viikonpäivää ja kellonaikaa, junatyyppejä, aikataulun mukaista matka-aikaa sekä matkustajapysähdysten lukumäärää.

Label on binäärinen: myöhässä (1) tai ajallaan (0), joka johdetaan pääaseman `differenceInMinutes`-arvosta. Jos juna saapuu vähintään viisi minuuttia aikataulusta jäljessä, se on myöhässä (1). Muuten se on ajallaan (0).

Käytämme Fintrafficin ylläpitämää Digitraffic-rautatieliikenne-rajapintaa, joka on avointa strukturoitua, taulukkomuotoista havainnointidataa. Haemme siitä junien aikataulu- ja toteumatiedot /trains-rajapinnan kautta. Rajapinnasta on saatavilla tiedot noin viimeiseltä kahdelta vuodelta, mutta käytämme datapisteinä yhden vuoden (2025) aikaisia tietoja. Esimerkiksi <https://rata.digitraffic.fi/api/v1/trains/2025-08-02>. Datan piirteet ovat sekä kategorisia (esim. junatyyppi, viikonpäivä) että jatkuvia (esim. reitin pituus), ja ennustettava label on binäärinen (myöhässä/ajallaan).

3. Metodit

Rajasimme aineiston vuoden 2025 matkustajaliikenteen kaukojuniin, jotta datapisteiden määrä pysyisi hallittavana ja laatu relevanttina ongelmaan nähden. Datapisteitä on esikäsittelyn jälkeen noin 71 000.

Esikäsittely (preprocessing):

1. Poistimme /trains-datasta kaikki muut junat paitsi kaukoliikenteen matkustajajunat (`trainCategory` muu kuin "Long-distance", tai "`trainType`": MV, V tai MUS)
2. Poistimme peruutetut ("`cancelled`": `true`) junat.
3. Poistimme jokaiselta junalta tiedot asemista, joilla se ei pysähtynyt ollenkaan tai joilla ollut matkustajapysähdystä ("`trainStopping`": `false` ja "`commercialStop`": `false`)
4. Poistimme tiedot yksittäisistä pysähdyksistä (lähtö- ja pääteasemaa lukuunottamatta) ja korvasimme ne niiden lukumäärällä (`numStops`)

Valitsimme piirteet, jotka olivat suoraan tai helposti johdettavissa raakadatasta ja jotka olivat saatavilla jo junan lähtöhetkellä. Tarkistimme myös alustavasti muutaman piirteen (kuten. junatyypin) yhteyttä myöhästymiseen datasta ennen lopullista valintaa. Esimerkiksi IC-junista noin 15 % oli myöhässä, kun taas HDM-junille vastaava luku on alle 5 %.

Lopulliset piirteet:

- Lähdön myöhästymä (`differenceInMinutes` lähtöasemalla)
- Junatyyppi (`trainType`)
- Viikonpäivä ja kellonaika (`scheduledTime`)
- Matka-aika (pääteaseman ja lähtöaseman `scheduledTime`-arvojen erotus)
- Matkustajapysähdysten lukumäärä (`numStops`)

Valitsimme logistic regression -mallin, koska:

- Se on nopea kouluttaa ja helposti tulkittavissa.
- Se antaa suoraan todennäköisyyksiä (esim. "65 % todennäköisyys myöhästyä"), mikä on hyödyllistä tässä sovelluksessa.
- Se toimii hyvänä baseline-mallina, johon voi myöhemmin verrata toista mallia.

Käytämme logistic loss -funktiota (binäärinen cross-entropy), sillä se on standardi valinta binääri luokitteluun logistic regressionilla. Se mittaa eron todellisen labelin ja mallin ennustaman todennäköisyyden välillä ja rankaisee mallia sitä enemmän mitä varmemmin väärä ennuste on. Minimoimalla tätä koulutuksen aikana malli säättää painokertoimia, jotta ennustetut todennäköisyydet vastaisivat todellisia myöhästymistapauksia.

$$H(p, q) = - \sum_{x \in \mathcal{X}} p(x) \log q(x). \quad (1)$$

Jaoin datan koulutus-, validointi- ja testijoukkoihin suhteessa 70/15/15 käyttäen satunnaista jakoa. Päädyimme tähän jakoon, sillä aineisto on riittävän suuri siihen, että 15 % siitä riittää antamaan luotettavan arvion mallin suorituskyvystä, ja suurin osa (70 %) jaa kouluttamiseen. Käytämme jaossa kiinteää random seediä, jotta jako on toistettavissa.

Tekoälyn käyttö

Tekoälyä on käytetty tässä vaiheessa datan esikäsittelyyn: sen avulla on kirjoitettu Python-skriptit datan siivoamiseen. Python-skriptejä on kirjoitettu myös laskemaan tiettyjen kenttien, kuten junatyypin, vaikutusta myöhästymiseen. Tämä teki esikäsittelystä ja datan tutkimisesta nopeaa ja mahdollisti eri vaihtoehtojen vaivattoman kokeilun.

Lähteet

Fintraffic, Rautatieliikenne. Saatavilla: <https://www.digitraffic.fi/rautatieliikenne/> (Lainattu: 16.9.2026).

Liitteet

Linkki GitHub-repoon, josta löytyy esikäsittelyssä käytetyt Python-skriptit sekä esikäsitelty data:
<https://github.com/piu-pau/TrainDelay>